



Carnegie Mellon University
School of Computer Science
Executive Education

Module 7: Capstone Project

Self-Study Capstone Checkpoint Activity 7.1: Final Capstone Planning

Final capstone report planning

1. **Project title**

Write a clear and descriptive title for your capstone project.

2. **Problem and user**

State the real-world problem your autonomous system addresses. Identify the intended user or users, and explain why this problem matters.

3. **System goal and scope**

Explain what your system is designed to do, what successful performance looks like, and any important boundaries or constraints you defined.

4. **Final system architecture**

Describe the final design of your system and how its major components work together. Depending on your project, this may include reasoning loops, memory, tools, retrieval, Tree-of-Thought reasoning, multi-agent coordination, guardrails, logging, evaluation, and human intervention.

5. Design evolution across the program

Review your earlier checkpoints and explain how your system evolved from Module 1 through Module 6. Highlight the most important refinements, additions, or changes, and explain why they improved the system.

6. Implementation overview

Summarize how you built the system. Include the frameworks, tools, models, application programming interfaces (APIs), or libraries you used, and explain how they support your design.

7. Evaluation and results

Explain how you evaluated your system. Describe the criteria, tests, or examples you used to assess performance, reliability, and limitations. Summarize the main results.

8. Safety and reliability considerations

Explain the guardrails, monitoring, fallback logic, and human oversight you included to improve safety and dependability.

9. **Limitations and next steps**

Identify the current limitations of your system and suggest realistic next steps for improving it.

10. **Public GitHub repository**

Prepare a public GitHub repository for your capstone project, and include the link in your final report. Your repository should make it possible for a technical audience to understand your project and review your implementation. At a minimum, it should include:

1. A README that explains the project, problem, architecture, setup steps, and usage
2. The main code or notebooks for your system
3. Any sample inputs, outputs, or evaluation artifacts you want to share
4. Clear instructions for running or reviewing the project